

1. **Who are you, what may you do:** Authentication vs authorisation · Ownership and roles: a valid token is not a free pass
2. **Passwords:** Store a hash, never the password · Salt and cost: why bcrypt is slow on purpose · Register and log in, step by step
3. **Sessions and tokens:** Remembering who you are: sessions vs tokens · Cookies and their security flags · Anatomy of a JWT · Signing and verifying: what the secret does · Sending, storing and expiring tokens · The auth middleware: one gate in front of many routes
4. **CORS:** Origins and the same-origin policy · CORS: the server says who may read · Preflight requests · Cookies across origins: credentials · What CORS is not
5. **Hardening the API:** Validation: never trust req.body · Security middleware: helmet, rate limits, body limits, HTTPS · Secrets live in environment variables · Pagination limits as protection · The security mistakes students actually make

Who are you, what may you do

Authentication vs authorisation

Authentication answers "who is calling?"; **authorisation** answers "may this caller do this, to this resource?". Failing the first gives 401; failing the second gives 403.

The two questions

	Authentication	Authorisation
Question	Who is calling?	May this caller do this to this resource?
Evidence	A password at log-in, then a token or session cookie	The user id and role from the verified token, plus the resource's owner
Where in an Express API	middleware/auth.js (sets req.user)	Each controller action (compares req.user.sub with task.user_id)
On failure	401 (missing, invalid or expired token)	403 (valid token, not yours or not allowed)

Example. A Tasks API's GET /tasks/7: with no Authorization header → **401** Missing token; with an expired token → **401**; with Ana's valid token when task 7 belongs to Ben → **403**; with Ana's token for her own task 7 → **200**; with any valid token for task 999, which does not exist → **404**.

Common mistake. Answering 403 for a missing token, or 401 for "not your task". The client reacts to each code differently: on 401 it sends the user to the log-in page; on 403 it shows "not allowed". Mixing them up sends logged-in users to a log-in page that cannot help them, and it is one of the most common mistakes in hand-written APIs.

Ownership and roles: a valid token is not a free pass

An ownership check compares the user in the verified token (req.user.sub) with the owner stored on the resource before reading or changing it; **a role check** asks whether the user's role (for example admin) permits the action at all.

```
// A notes API (the same pattern works for tasks): load, 404, then 403.
async function getNote(req, res, next) {
  try {
    const note = await notesModel.findById(Number(req.params.id));
    if (!note) return res.status(404).json({ error: 'Note not found' });
    if (Number(note.owner_id) !== Number(req.user.sub)) {
      return res.status(403).json({ error: 'This note is not yours' });
    }
    res.json(toDTO(note));
  } catch (err) {
    next(err);
  }
}

// A role check: a middleware factory used after auth.
const requireRole = (role) => (req, res, next) =>
  (req.user.role === role ? next() : res.status(403).json({ error: 'Forbidden' }));

router.delete('/:id', requireRole('admin'), deleteNote);
```

Example. Ana (sub: 1) sends PUT /notes/8; note 8 has owner_id: 2. Her token is perfect, so auth lets her through, and the controller answers **403**. She then sends POST /notes with the body {"title":"Hi", "userId":2}: the server ignores userId and stores the note with owner 1, the sub from her token.

Common mistake. Taking the owner from the request: create({ ...req.body }) or findByOwner(req.query.userId). Anything the client sends can be changed by the client; only the verified token says who is calling. A second classic: comparing ids with !== without converting them, so every owner gets 403 (or, with !=, getting lucky without understanding why).

Passwords

Store a hash, never the password

A **hash function** turns a password into a fixed-length fingerprint that cannot be turned back into the password; the server stores only the fingerprint and checks a log-in by hashing the attempt again and comparing.

Three ways to keep a password, from worst to right

	Plaintext	Encrypted	Password hash (bcrypt)
Stored value	hunter2	Ciphertext + a key on the server	\$2b\$10\$... (60 characters)
Can it be turned back?	It is the password	Yes, by anyone with the key	No: only guessed
Log-in check	Compare strings	Decrypt and compare	Hash the attempt and compare (bcrypt.compare)
After a database leak	Every password is known	Every password, if the key leaks too	Each password must be guessed, slowly

Example. SHA-256 of hunter2 is f52fbd32...26a3f6c7; of hunter3 it is fb8c2e2b...21cfa57a: one character apart, nothing in common. A site whose "forgot password" email sends you **your old password** is storing it reversibly; the right flow emails a one-time reset link instead.

Common mistake. "Encrypting passwords with a secret key is safer than hashing." It is worse: the key has to live next to the data so the server can decrypt, and an attacker who gets the database usually gets the key too. Nobody, not even the server, ever needs to read a password back.

Salt and cost: why bcrypt is slow on purpose

A **salt** is a random value stored with each hash so that equal passwords get different hashes; a **cost factor** makes each hash deliberately slow, so an attacker can try far fewer guesses per second.

```
const bcrypt = require('bcrypt'); // npm install bcrypt (bcryptjs has the same API)

// sign-up: a new random salt every time
const hash = await bcrypt.hash('correct horse battery staple', 10);
// e.g. '$2b$10$N9qo8uLOickgx2ZMRZoMyeIjZAgcfl7p92ldGxad68LJZdL17lhwy'
// '$2b$' version · '10$' cost · then 22 chars of salt and 31 chars of hash

// log-in: re-hash the attempt with the salt and cost stored in the hash
await bcrypt.compare('correct horse battery staple', hash); // true
await bcrypt.compare('Correct horse battery staple', hash); // false
```

Example. Two users both choose 123456. Unsalted SHA-256 stores the same value twice, 8d969eef6ecad3c2..., and a web search for it reveals the password: crack one, crack both. With bcrypt the two rows look completely unrelated. In the tool below, hash a password at 2¹⁶ and then 2¹⁸ iterations: the time roughly quadruples, and so does the attacker's.

Common mistake. Thinking the salt must be kept secret, or storing it "safely" elsewhere. A salt is not a secret: its only job is to be **different for every user**. Using one fixed salt for everybody defeats that job, because equal passwords become equal hashes again.

Example. A forum shows comments with innerHTML, and someone posts ``. Without `HttpOnly`, every reader's session id is sent to the attacker, who can then act as them. With `HttpOnly`, `document.cookie` does not contain `sid` and the theft fails (the real fix is still to stop the XSS).

Common mistake. Thinking `HttpOnly` makes a cookie "safe". It stops scripts from **reading** it, but the browser still **sends** it, so CSRF is still possible (that is SameSite's job), and an XSS can still make requests as the user from inside the page.

Anatomy of a JWT

A **JSON Web Token** is three `base64url` strings joined by dots, `header.payload.signature`: the header names the signing algorithm, the payload holds the **claims** (statements such as the user id and the expiry time), and the signature proves that the first two have not been changed.

Claims you will meet (a typical log-in token has `sub`, `role`, `iat` and `exp`)

Claim	Name	Meaning	Who sets it
<code>sub</code>	Subject	Who the token is about: the user id. Becomes <code>req.user.sub</code> .	You: <code>jwt.sign({ sub: user.id, ... })</code>
<code>iat</code>	Issued at	When it was signed (Unix seconds).	<code>jwt.sign</code> , automatically
<code>exp</code>	Expiration	After this moment <code>jwt.verify</code> throws <code>jwt expired</code> .	<code>jwt.sign</code> with <code>expiresIn: "1h" → iat + 3600</code>
<code>nbf</code>	Not before	Not valid before this moment.	Optional (<code>notBefore</code>)
<code>iss / aud</code>	Issuer / audience	Who issued it / who it is meant for.	Optional
<code>role</code>	(custom)	Your own claim: lets routes check permissions without a database lookup.	You

Example. The example token on `jwt.io` starts `eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9`, which decodes to the header `{"alg":"HS256","typ":"JWT"}`. Its payload is `{"sub":"1234567890","name":"John Doe","iat":1516239022}`: `iat` is 18 January 2018, 01:30:22 UTC. It has no `exp`, so it never expires, which you would never do in a real API. Paste it into the inspector below and change one character.

Common mistake. Putting secrets in the payload ("it is a token, so it is hidden"): a password hash, another user's data, an internal API key. Anyone who sees the token can decode it in one line. A second classic: `exp: Date.now() + 3600`, which is in **milliseconds**, so the token expires in about 56,000 years.

Signing and verifying: what the secret does

With **HS256**, `jwt.sign` computes an **HMAC-SHA-256** of `header.payload` using a secret that only the server knows; `jwt.verify` recomputes it and rejects the token if the result differs, if the algorithm is not the expected one, or if the token has expired.

```
const jwt = require('jsonwebtoken');

// at log-in
const token = jwt.sign({ sub: user.id, role: user.role }, process.env.JWT_SECRET, { expiresIn: '1h' });

// on every protected request
try {
  const payload = jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256'] });
  // payload → { sub: 42, role: 'user', iat: 1791158400, exp: 1791162000 }
} catch (err) {
  // err.name → 'TokenExpiredError' | 'NotBeforeError' | 'JsonWebTokenError'
}

// A strong random secret (run once, put the output in .env):
// node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Example. Ana decodes her token, changes "role":"user" to "role":"admin", encodes it again and sends it. The server recomputes HMAC-SHA-256 over the new header and payload with its secret; the result does not match the old signature still attached, so `jwt.verify` throws `invalid signature` and the middleware answers **401**. Try it with the **Tamper** button of the inspector.

Common mistake. Using `jwt.decode(token)` in the middleware instead of `jwt.verify(token, secret)`. `decode` only reads the payload and checks nothing, so anyone can write a token saying `role: admin`. Another: a short secret such as `secret` or `riverside123`. Anyone holding one token can try millions of candidate secrets offline until the signature matches; use 32 random bytes.

Sending, storing and expiring tokens

The client sends the token on every protected request in the `Authorization: Bearer <token>` header; it has to keep it somewhere in between, and the `exp` claim limits how long a stolen copy stays useful.

```
// Front end: log in, keep the token, send it back
const login = await fetch('http://localhost:3000/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password }),
});
if (!login.ok) throw new Error('Wrong email or password');
const { token } = await login.json();

const res = await fetch('http://localhost:3000/tasks', {
  headers: { Authorization: `Bearer ${token}` },
});
// From a terminal: curl -H "Authorization: Bearer $TOKEN" http://localhost:3000/tasks
```

Example. A token signed at 10:00:00 with `expiresIn: "1h"` has `exp = 11:00:00`. A request at 10:59:59 passes; at 11:00:00 exactly `jwt.verify` throws `jwt expired` (the check is `now ≥ exp`), the API answers **401**, and the front end should send the user back to the log-in form. Shift the clock in the inspector to watch it happen.

Common mistake. Getting the header format wrong: `Authorization: eyJ...` (no `Bearer`), `Bearer: eyJ...` (a header called `Bearer`), or `?token=eyJ...` in the URL. The middleware in the next card only accepts a value that starts with `Bearer` followed by the token, and treats anything else as a missing token (**401**).

The auth middleware: one gate in front of many routes

An **auth middleware** runs before the protected routes: it reads the `Authorization` header, verifies the token, stores the decoded payload in `req.user` and calls `next()`, or stops the request with **401**.

Three requests to `GET /tasks`

Request header	What auth does	Result
(none)	token is null	401 Missing token; the router never runs
<code>Authorization: Bearer abc</code>	<code>jwt.verify</code> throws <code>jwt malformed</code>	401 Invalid or expired token
<code>Authorization: Bearer eyJ...</code> (valid)	<code>req.user = { sub: 42, role: 'user', iat, exp },</code> then <code>next()</code>	The tasks controller runs with <code>req.user</code>

```
// middleware/auth.js
const jwt = require('jsonwebtoken');

function auth(req, res, next) {
  const header = req.headers.authorization || ''; // Node lower-cases header names
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ error: 'Missing token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET, { algorithms: ['HS256'] });
    next();
  } catch (err) {
    res.status(401).json({ error: 'Invalid or expired token' });
  }
}
```

```

}

// app.js
app.use('/auth', authRouter);           // open: this is where tokens come from
app.use('/tasks', auth, tasksRouter);   // every /tasks route is behind the gate

```

Example. Ana's React page calls GET /tasks?page=2 with her token. auth verifies it and sets req.user.sub = 1; the list controller then asks the model for **her** tasks only (findByOwner(1, { limit, offset })). Ben's token on the same URL gives Ben's tasks: same route, different req.user.

Common mistake. Forgetting return before res.status(401).json(...). The function keeps going, reaches next() and the protected route runs anyway, usually followed by the error "Cannot set headers after they are sent to the client". Calling next() inside the catch has the same effect.

CORS

Origins and the same-origin policy

An **origin** is the scheme + host + port of a URL; the browser's **same-origin policy** lets a page's JavaScript read responses freely only from its own origin.

Compared with <http://localhost:3000/api/tasks>

URL	Same origin?	Why
http://localhost:3000/index.html	Yes	Only the path differs
http://localhost:5173/	No	Port 5173 ≠ 3000
https://localhost:3000/	No	Scheme https ≠ http
http://127.0.0.1:3000/	No	Host 127.0.0.1 ≠ localhost (same machine, different name)
http://localhost/	No	No port written means 80 ≠ 3000

Example. The page at <http://localhost:5173> runs fetch('http://localhost:3000/api/tasks'). The ports differ, so it is cross-origin. With no CORS configured, the Express terminal still logs GET /api/tasks: the request arrived and the route ran. But the browser console shows a CORS error and the page's fetch rejects with TypeError: Failed to fetch.

Common mistake. "The same-origin policy stops the request from reaching the server." For simple requests it does not: the server receives and executes them, and only the response is hidden. That is one more reason why a GET must never change data, and why CORS settings are no replacement for authentication.

CORS: the server says who may read

CORS (Cross-Origin Resource Sharing) is a set of HTTP response headers with which a server tells the browser which other origins may read its responses; the key one is Access-Control-Allow-Origin.

```

const cors = require('cors');           // npm install cors

app.use(cors({ origin: 'http://localhost:5173' })); // before the routers (and before auth)
app.use(express.json());
app.use('/api/tasks', tasksRouter);

// The exchange it produces:
// GET /api/tasks HTTP/1.1
// Origin: http://localhost:5173
//
// HTTP/1.1 200 OK
// Access-Control-Allow-Origin: http://localhost:5173
// Vary: Origin           ← caches must keep one copy per Origin
// Content-Type: application/json; charset=utf-8

```

Example. With `cors({ origin: 'http://localhost:5173' })`, the React page reads the tasks. A page on `https://evil.example` making the same call gets `Access-Control-Allow-Origin: http://localhost:5173`, which does not match its own origin, so the browser blocks it with the message "...has a value 'http://localhost:5173' that is not equal to the supplied origin". Try both in the simulator.

Common mistake. Adding `Access-Control-Allow-Origin` to the **request** in the front-end code: `fetch(url, { headers: { 'Access-Control-Allow-Origin': '*' } })`. It is a **response** header that only the server can grant; on a request it does nothing useful, and as a non-safelisted header it even triggers a preflight.

Preflight requests

Before a cross-origin request that is not "simple" (for example a PUT, a DELETE, a JSON body or an Authorization header), the browser first sends an OPTIONS preflight asking for permission, and sends the real request only if the answer allows it.

- A preflight is needed when the method is not GET, HEAD or POST,
- or a header other than Accept, Accept-Language, Content-Language and Content-Type is sent (Authorization always counts),
- or Content-Type is anything other than the three form types (application/json counts).
- Cookies (credentials) do **not** cause a preflight on their own.

HTTP

```
OPTIONS /api/tasks/7 HTTP/1.1
Host: localhost:3000
Origin: http://localhost:5173
Access-Control-Request-Method: DELETE
Access-Control-Request-Headers: authorization

HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
Access-Control-Allow-Headers: authorization
Access-Control-Max-Age: 600
Vary: Origin, Access-Control-Request-Headers
```

Example. The React page runs `fetch('http://localhost:3000/api/tasks/7', { method: 'DELETE', headers: { Authorization: 'Bearer ...' } })` against `cors({ origin: 'http://localhost:5173', methods: ['GET', 'POST'] })`. The preflight answer lists only GET,POST, so Chrome logs "Method DELETE is not allowed by Access-Control-Allow-Methods in preflight response.", the DELETE is never sent and the task is still there.

Common mistake. Mounting the auth middleware before `cors`. The browser never sends Authorization on a preflight, so auth answers the OPTIONS request with 401; a preflight that is not 2xx fails, and every protected call from the front end shows a CORS error even though the token is fine. Put `app.use(cors(...))` first.

Cookies across origins: credentials

By default fetch does not send cookies to another origin; to send them the page must ask with `credentials: 'include'`, **and the server must answer with an exact origin (never *) plus** `Access-Control-Allow-Credentials: true`.

```
// Server
app.use(cors({ origin: 'http://localhost:5173', credentials: true }));

// Page
const res = await fetch('http://localhost:3000/api/me', { credentials: 'include' });
```

Example. With plain `cors()` and `credentials: 'include'`, Chrome logs: "The value of the 'Access-Control-Allow-Origin' header in the response must not be the wildcard '*' when the request's credentials mode is 'include'." Changing the server to an exact origin and `credentials: true` fixes it. Challenge 5 of the simulator is exactly this.

Common mistake. `cors({ origin: '*', credentials: true })`. The cors package sends exactly what you asked for, * together with `Allow-Credentials: true`, and the browser rejects that combination, so nothing works and the configuration looks "correct" in the code.

What CORS is not

CORS is enforced *only by browsers* and only controls which web pages may read responses; it does not identify anyone, and it does not stop curl, Postman, scripts or other servers from calling your API.

Who CORS applies to

Client	Sends Origin?	Checks CORS headers?	Can call your API?
Your React page (another origin)	Yes	Yes	Yes, and reads the answer if CORS allows it
A page on another site	Yes	Yes	Simple requests are sent and run; reading is blocked
curl / Postman / a script	No	No	Yes, always
Another server (Node fetch)	No	No	Yes, always

Example. Your API uses `cors({ origin: 'http://localhost:5173' })` but forgot the auth middleware on `DELETE /api/tasks/:id`. Anyone can run `curl -X DELETE http://localhost:3000/api/tasks/7` and the task is gone: the CORS setting played no part.

Common mistake. "I restricted CORS to my front end, so only my front end can use my API." Anyone can still call it outside a browser, and even inside a browser other sites' simple requests reach the server. Only authentication and authorisation decide who may do what.

Hardening the API

Validation: never trust `req.body`

Input validation checks every value that arrives over HTTP (body, URL parameters, query string) against a declared shape before your code uses it, and rejects bad input with `400` and a list of what is wrong.

```
const { z } = require('zod');

const noteSchema = z.object({
  title: z.string().trim().min(1).max(120),
  pinned: z.boolean().default(false),
});

noteSchema.safeParse({ title: '', pinned: 'yes', userId: 7 });
// → { success: false, error } with error.issues:
//   [ { code: 'too_small', path: ['title'], message: 'String must contain at least 1 character(s)', ... },
//     { code: 'invalid_type', path: ['pinned'], message: 'Expected boolean, received string', ... } ]

noteSchema.safeParse({ title: 'Buy milk', userId: 7 });
// → { success: true, data: { title: 'Buy milk', pinned: false } }  userId stripped, default added
```

Example. In a Tasks API with a `validate(taskSchema)` middleware, `POST /tasks` with `{ "done": true }` (no title) never reaches the controller: `validate(taskSchema)` answers **400** `{ "error": [ { "code": "invalid_type", "path": [ "title" ], "message": "Required", ... } ] }`. With `{ "title": "Buy milk", "userId": 99 }` it passes, `req.body` becomes `{ title: 'Buy milk', done: false }`, and the controller sets the owner from `req.user.sub`.

Common mistake. Validating and then carrying on with the original `req.body` instead of the parsed data: the stripped fields (`userId`, `isAdmin`) come straight back. Another: validating `POST` but forgetting `PUT` and `PATCH`, which write to the same database.

Security middleware: helmet, rate limits, body limits, HTTPS

A few lines of middleware close whole classes of holes: **helmet** sets protective response headers, **express-rate-limit** caps how many requests one client may send, `express.json({ limit })` caps body size, and HTTPS with

HSTS keeps passwords and tokens from crossing the network in plain text.

What each one protects against

Middleware	Protects against	When it triggers
helmet()	MIME sniffing, clickjacking, some XSS, revealing that you run Express	Adds headers to every response
rateLimit(...)	Password guessing, scrapers, one client flooding the API	429 Too Many Requests
express.json({ limit })	Huge bodies that exhaust memory	413 Payload Too Large
cors({ origin })	Other web pages reading your API in a browser	Missing CORS headers → the browser blocks
HTTPS + HSTS	Passwords and tokens read on the network	Always (HSTS: browser refuses plain http)

```
const helmet = require('helmet');
const rateLimit = require('express-rate-limit');

app.use(helmet());
app.use(express.json({ limit: '100kb' })); // the default: keep it small
app.use('/auth/login', rateLimit({ windowMs: 15 * 60 * 1000, limit: 10 })); // 10 tries / 15 min / IP
app.use(rateLimit({ windowMs: 60 * 1000, limit: 100 })); // 100 requests / min / IP
// express-rate-limit 7 calls the option `limit`; older versions and many tutorials call it `max`.
```

Example. An attacker scripts 10,000 log-in attempts for ana@example.com. With the log-in limiter, attempt 11 within 15 minutes gets **429**; at 10 tries per quarter of an hour, 10,000 guesses take more than ten days, and bcrypt makes each one slow on top. Without it, the only limit is your server's CPU.

Common mistake. Thinking helmet() "secures the API". It only sets response headers: it does nothing against broken access control, injection or a leaked secret. Security middleware is a seatbelt, not a substitute for driving carefully.

Secrets live in environment variables

A secret (the JWT signing key, the database password, API keys) is never written in the code or committed to Git; the app reads it from an environment variable such as process.env.JWT_SECRET, set outside the code.

```
// .env (in .gitignore, never committed)
// JWT_SECRET=3f9c2b7e0d8a41f6b5c3e9a7d2f1084c6b5a3e2d1f0c9b8a7e6d5c4b3a291807

// config.js
require('dotenv').config(); // or start Node 20.6+ with: node --env-file=.env src/server.js
const isProduction = process.env.NODE_ENV === 'production';
if (isProduction && !process.env.JWT_SECRET) {
  throw new Error('JWT_SECRET is missing'); // crash at start-up, not on the first log-in
}
module.exports = { jwtSecret: process.env.JWT_SECRET || 'dev-secret-change-me' };
```

Example. A pair pushes const JWT_SECRET = 'riverside123'; to a public repository. Anyone can read it, sign a token saying role: admin and delete every fixture. The fix: generate a new random secret, put it in the server's environment, remove it from the code, and accept that every user has to log in again.

Common mistake. Committing .env "just once" and deleting it in the next commit, believing the problem is gone. It is still in the history. Another: logging process.env or the whole config at start-up, which prints the secrets into log files.

Pagination limits as protection

A list endpoint returns results in pages chosen by ?page= and ?limit=, and the server clamps both to safe bounds, so no caller can make it read and send an unbounded number of rows.

```
function clampPage(query) {
  const page = Math.max(1, Math.floor(Number(query.page)) || 1);
  const limit = Math.min(100, Math.max(1, Math.floor(Number(query.limit)) || 20));
  return { page, limit, offset: (page - 1) * limit };
}
```

```

console.log(clampPage({})); // nothing sent: defaults
console.log(clampPage({ page: '3', limit: '10' })); // query values are strings
console.log(clampPage({ page: '0', limit: '999' })); // both clamps at work
console.log(clampPage({ page: '-4', limit: '-1' })); // negative values
console.log(clampPage({ page: 'abc', limit: '2.7' })); // junk and decimals

// A common snippet with no lower bound for limit:
const naiveLimit = (q) => Math.min(100, Number(q.limit) || 20);
console.log(naiveLimit({ limit: '-1' })); // -1 would reach SQL as LIMIT -1

```

Example. GET /tasks?limit=999999 gives 100 items, ?page=0 is treated as page 1, and ?limit=abc falls back to 20. With the naive snippet, ?limit=-1 passes -1 to the database: PostgreSQL rejects LIMIT -1 with an error (a 500 for the client) and SQLite treats it as "no limit at all".

Common mistake. Clamping only the upper bound (as many tutorial snippets do), or using req.query.limit directly. Remember that Number('0') || 20 is 20 (0 is falsy) and that Number('-5') is a perfectly truthy -5.

The security mistakes students actually make

Most breaches of small APIs come from a handful of mistakes that the **OWASP Top 10** lists every few years: broken access control, injection, exposed secrets, weak authentication and careless configuration. Each one has a known, simple defence.

Risk → how it looks → defence

Risk	In a small API	Defence	Where
Broken access control	GET /tasks/8 returns another user's task; the owner comes from the body	Ownership check with the owner from the token; filter lists by owner	Ownership card
Injection (SQL)	String concatenation in a query	Parameterised queries (\$1, ?)	Relational databases
Injection (NoSQL)	{"email":{"\$ne":null}} passed into a MongoDB find	Validate types with a schema	Validation card; Document databases
Cross-site scripting (XSS)	API data inserted with innerHTML	textContent; React escapes by default	DOM and events
Exposed secrets / data	JWT_SECRET in Git; stack traces or password_hash in responses	Environment variables, rotation, generic error messages, toDTO	Secrets card; Error handling (Routes and middleware)
Authentication failures	Plaintext or fast hashes, no rate limit, tokens without exp, jwt.decode, alg: none	bcrypt, a log-in limiter, expiresIn, jwt.verify with algorithms	Passwords and Tokens groups
Security misconfiguration	cors({ origin: true, credentials: true }), X-Powered-By, debug output	Exact CORS origins, helmet	CORS group; security middleware
Vulnerable dependencies	Old packages with known holes	npm audit, keep package-lock.json, update	Server-side JavaScript

Example. A quick self-audit of a Tasks API: log in as Ana and request Ben's task (expect **403**); send a task with userId of someone else (expect your own id stored); send ?limit=-1 and ?limit=999999 (expect 1 and 100 items at most); search the repository for JWT_SECRET = (expect only process.env); trigger an error (expect no stack trace in the response).

Common mistake. "Security is something we add at the end." Some layers (helmet, rate limits) can be added at the edges, but the important decisions shape the code from day one: whether the owner comes from the token, whether passwords are hashed, whether errors leak details. Retrofitting them after users and data exist is far harder.